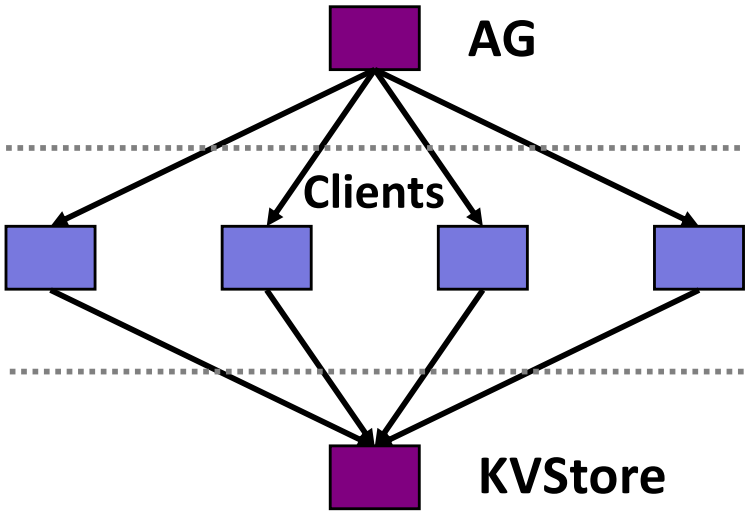
CompSci 390-03-26S

Distributed and Networked Systems

Let's Go: C in the Rearview Mirror

A distributed system in a process?

- Distributed systems are made up of processes, running on machines, communicating by sending/receiving messages over network channels.
- We can mirror their structure with actors (goroutines) sending messages over channels within a process.



A program runs as a process

- With a **virtual memory** (VM) aka Virtual Address Space.
- Containing **segments** initialized from the binary file.
- A binary program is a file in a special format, e.g., ELF.
- File **sections**: machine code, initial data, constant pool.

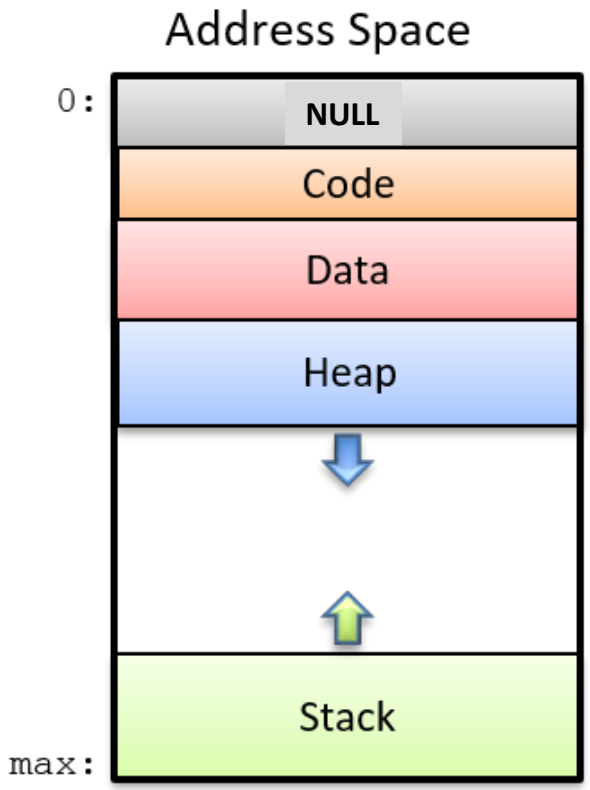
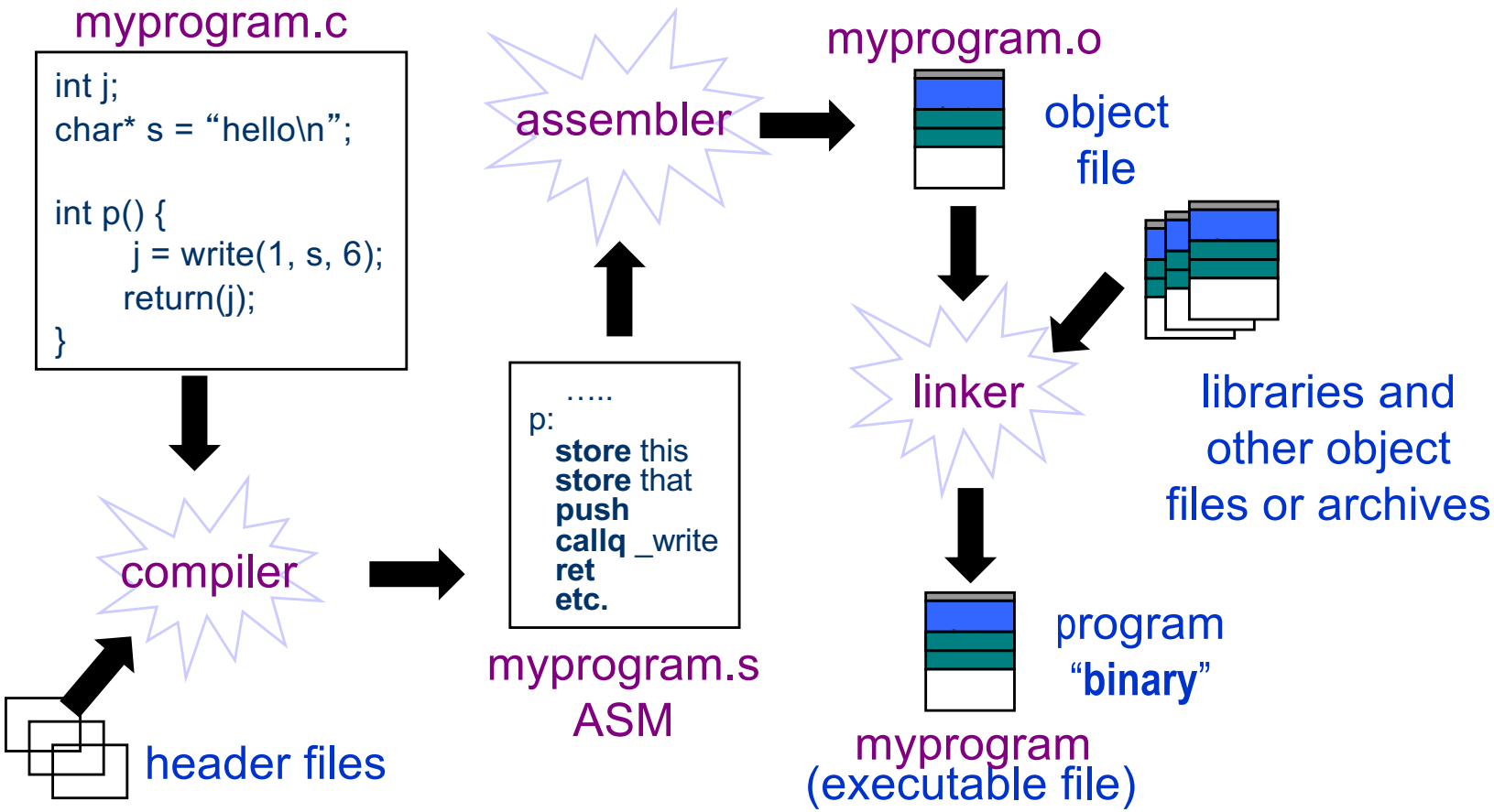


Image from diveintosystems.org

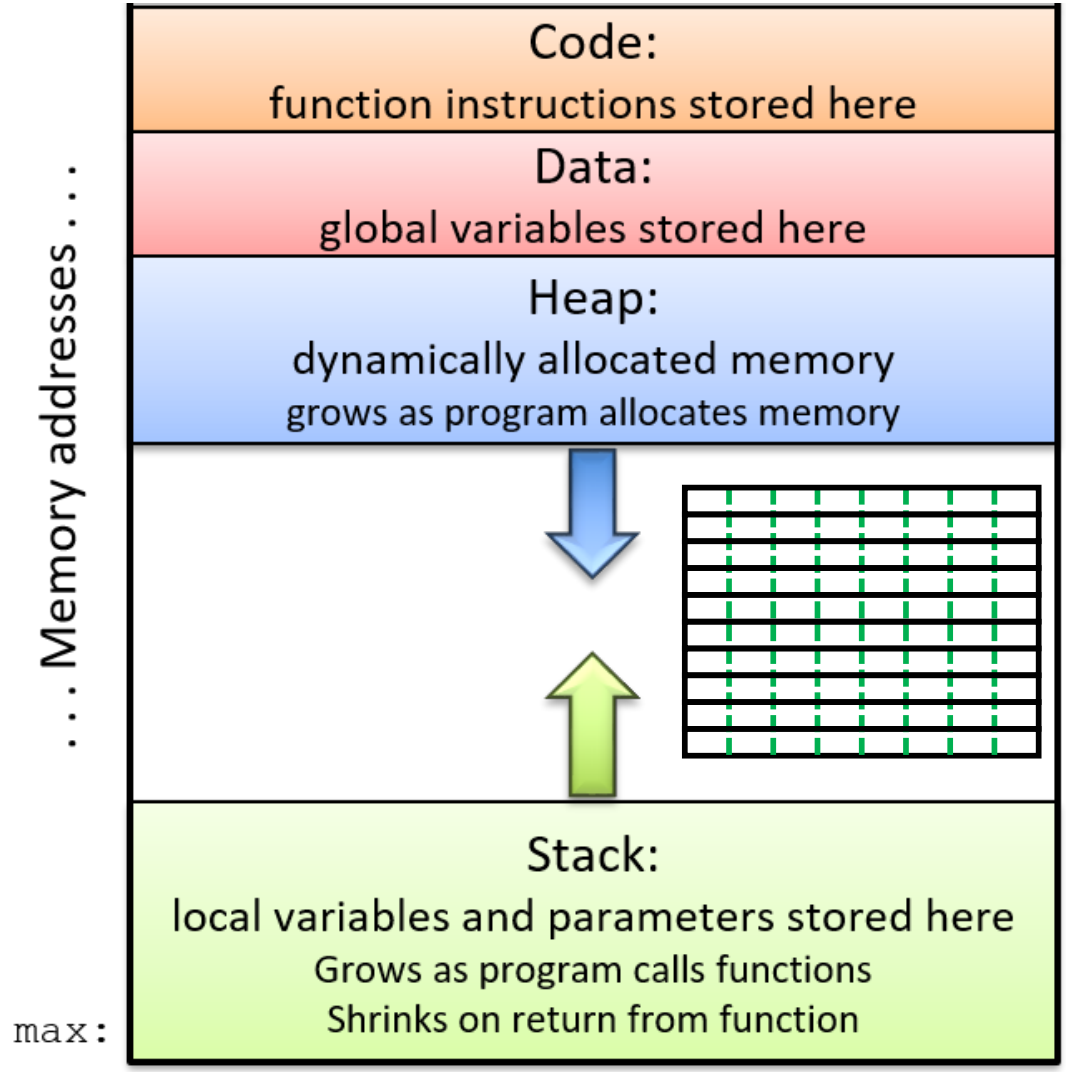
Virtual Memory is
- a set of segments
- an array of bytes

Birth of a program (C)



Where is your data?

- Linker and OS cooperate on process memory layout.
- Detailed in program binary.
- **Static data** (e.g., externals) go in Data region at launch.
- **Dynamic data**: allocate space at runtime: **stack, heap**.
- **Note**: we can draw the same picture "upside down", with max at the top.
- Layout varies. It's a cartoon!



[Image courtesy of diveintosystems.org]

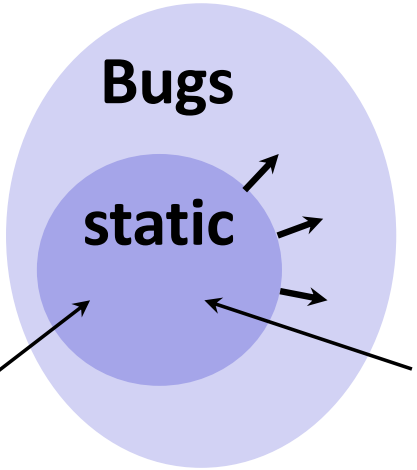
go run

- go shell command
- Compile source file
- Import its packages
- Compile packages as needed
- Link it all together
- Produce a binary (ELF on Linux)
- Run main() the usual way
- Delete it
- (The binary lives in /tmp...)

```
% go run echopipe.go
main: sending -> "Halloooo"
0: "Halloooo"
1: "Halloooo"
2: "Halloooo"
3: "Halloooo"


3: echoing...
2: "Halloooo"
1: "Halloooo"
0: "Halloooo"
main: received echo: "Halloooo"
```

Type checking and memory safety

- Can the compiler catch all errors statically?
- **No**. Some slip by and cause a mess: **bugs**!
- C is unsafe: **smash**, **trash**, and **crash**.
- Go property: **memory safety**
  - Strong static types
  - Runtime support
  - Runtime checks
- Runtime cost > C, or Rust

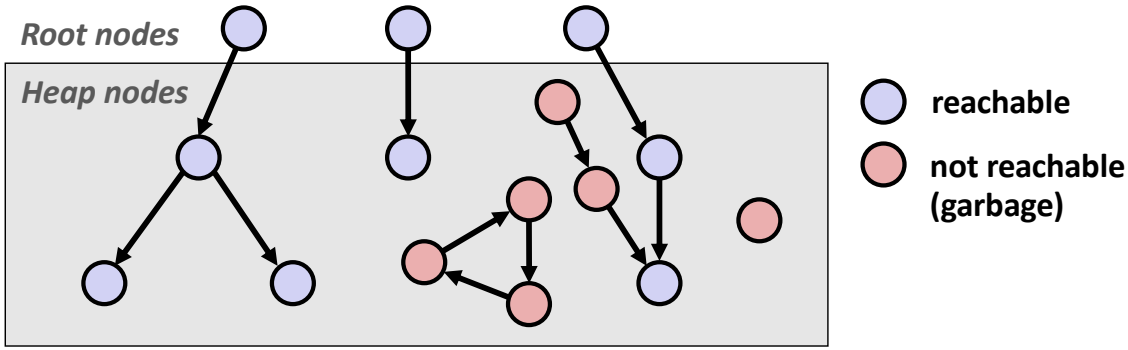


Bugs the compiler can detect statically

Research problem: grow this category. Requires new PL design!

Go has garbage collection

- Search the process data graph to determine reachability?
  - Must know the location and type of pointers in each block.
  - Must know values in roots (e.g., global variables), and their types.
  - Runtime tracking with help from the compiler.
  - At runtime: periodic CPU bursts for GC to clear garbage "exhaust".
- Safe GC requires discipline in the PL.
  - Knowable references
  - No untyped pointers!
  - No pointer arithmetic!
  - No pointers "into" a block!



Idiomatic constructor

```
func NewFile(fd int, name string) *File {
    if fd < 0 {
        return nil
    }
    f := File{fd, name, nil, 0}
    return &f
}
```

https://go.dev/doc/effective_go#allocation_new

Implicit declaration

????

Static **escape analysis** puts f on the heap

Composite literal

Function types: grammars and BNF

- Functions are “first-class objects” in Go: e.g., we can declare and assign variables of **function types** and use them in expressions.
- This snippet from the go spec expresses the syntax of function types.
- They are rules of a generative **context-free grammar** for legal Go syntax.
- Rules are written in a variant of **Backus-Naur Form (BNF)**.

```
FunctionType = "func" Signature .
Signature = Parameters [ Result ] .
Result = Parameters | Type .
Parameters = "(" [ ParameterList [ "," ] ] ")" .
ParameterList = ParameterDecl { "," ParameterDecl } .
ParameterDecl = [ IdentifierList ] [ "..." ] Type .
```

E.g.:
func (a int) int

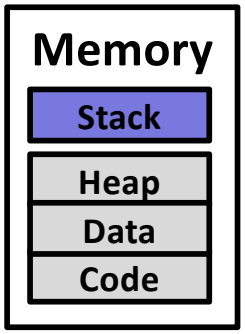
Maps

- Built-in associative type
- Aka **hash** or **dictionary**
- Use **make** statement to construct
- Access like an array
- **delete** statement removes entry
- Resizes automatically

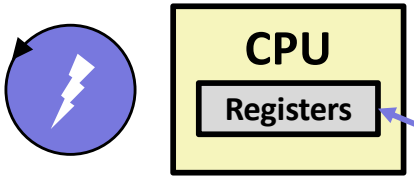
```
m := make(map[string]int)

m["k1"] = 7
m["k2"] = 13

fmt.Println("map:", m)

v1 := m["k1"]

delete(m, "k2")
```

https://gobyexample.com/maps

Deconstructing processes: threads



+



Thread of control

Register state: values in registers at execution point.

Virtual Address Space

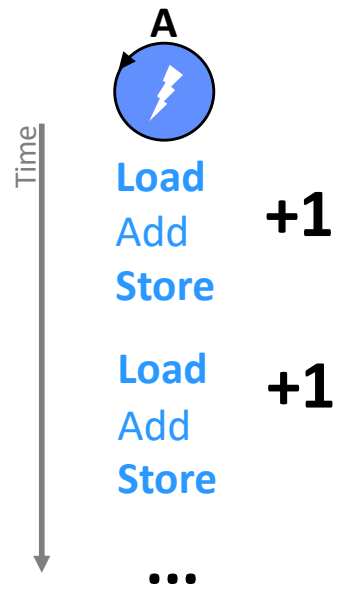
Mapped segments in memory

A **classic process** has a VAS and **one** thread, which starts in `main`(). But modern programs may create **more threads**!

- An execution **context**
- Executes a **sequence** of instructions
- At any point in time, the context is defined by one set of register values
- **Inhabits** an address space
- **Owns** a stack in its VAS—for call/return

Go's go statement: create a goroutine (thread)

```
var counter int

func count() {
        for i := 0; i < 10_000_000; i++ {
                counter = counter + 1
        }
}

func main() {
        go count()
        fmt.Printf("counter = %d\n", counter)
}
```



What does it print?

Waiting for threads to finish

```
func count(wg *sync.WaitGroup) {
        defer wg.Done()
        for i := 0; i < 10_000_000; i++ {
                counter = counter + 1
        }
}

func main() {
        var wg sync.WaitGroup
        wg.Add(1)
        go count(&wg)
        wg.Wait()
        fmt.Printf("counter = %d\n", counter)
}
```

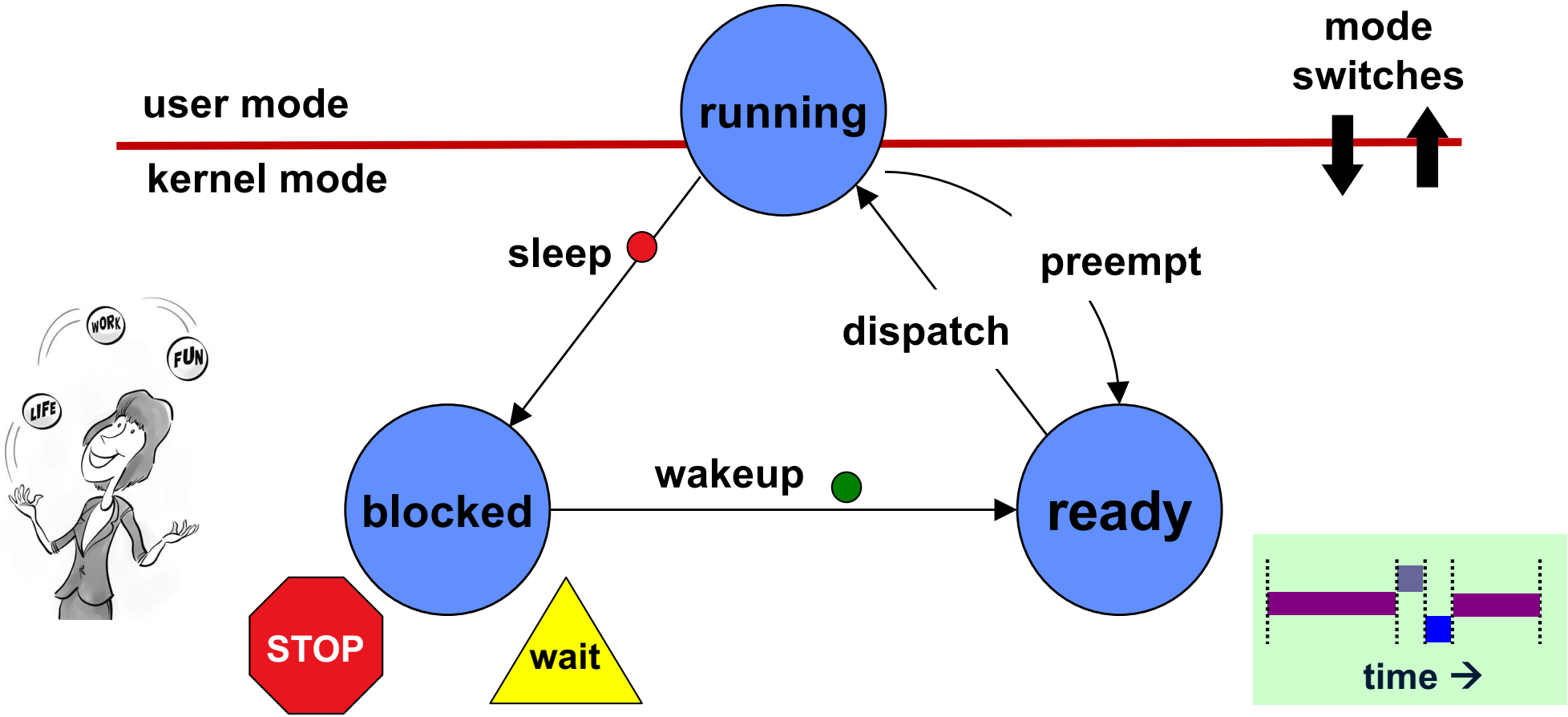
Defer: Do it at the end, no matter what

WaitGroup.Done(): report thread is done

WaitGroup: like thread join(), but for a group.

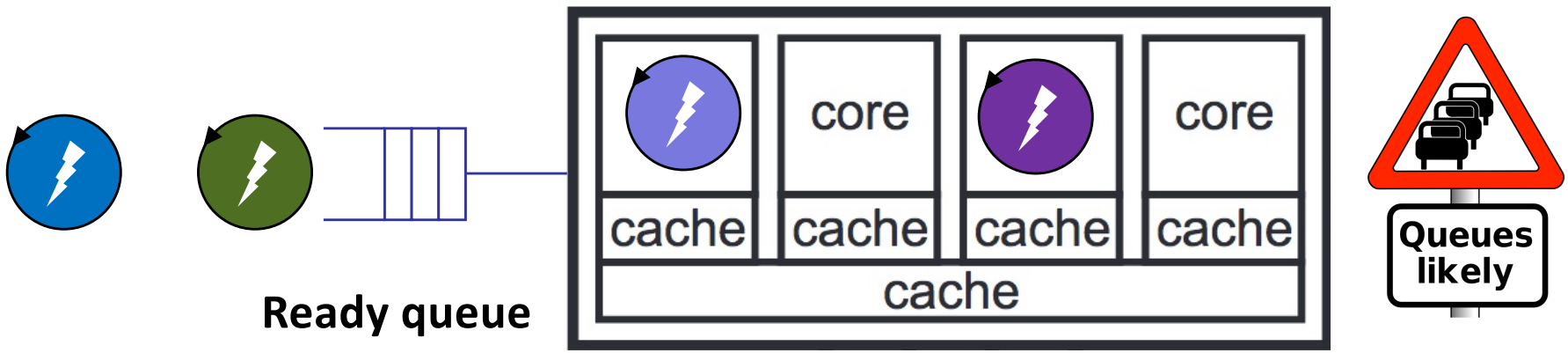
What does main() do while waiting?

Virtualizing the CPU: thread states



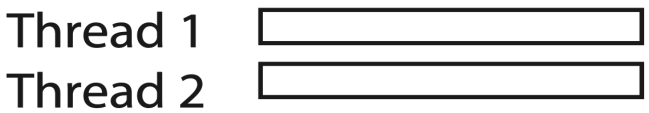
Threads and cores: pairing up

- Thread in the **ready/runnable** state?
- OS **dispatches** thread to an idle core.
- No idle core? Thread queues in **ready queue** (aka **run queue**).

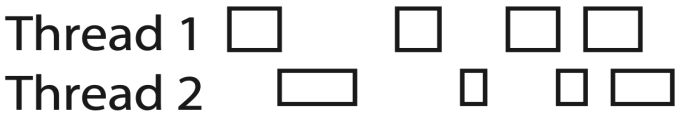


Non-deterministic schedules: possible executions

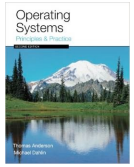
a) One execution



b) Another execution

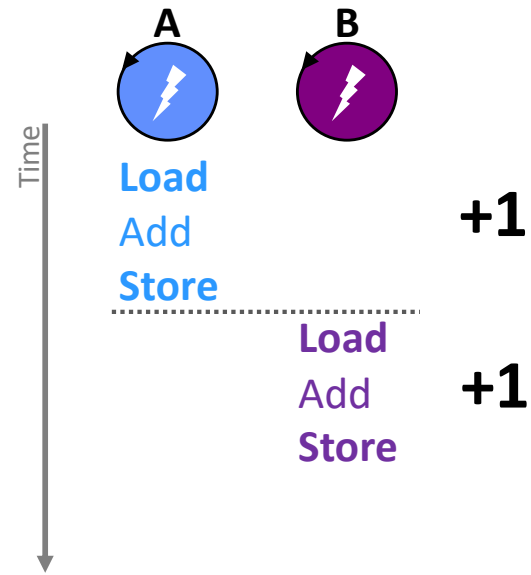


c) Another execution



Counter example in Go

```
func count(name string, wg *sync.WaitGroup) {
        defer wg.Done()
        for i := 0; i < 10_000_000; i++ {
                counter = counter + 1
        }
}

func main() {
        var wg sync.WaitGroup
        wg.Add(2)
        go count("A", &wg)
        go count("B", &wg)
        wg.Wait()
        fmt.Printf("counter = %d\n", counter)
}
```

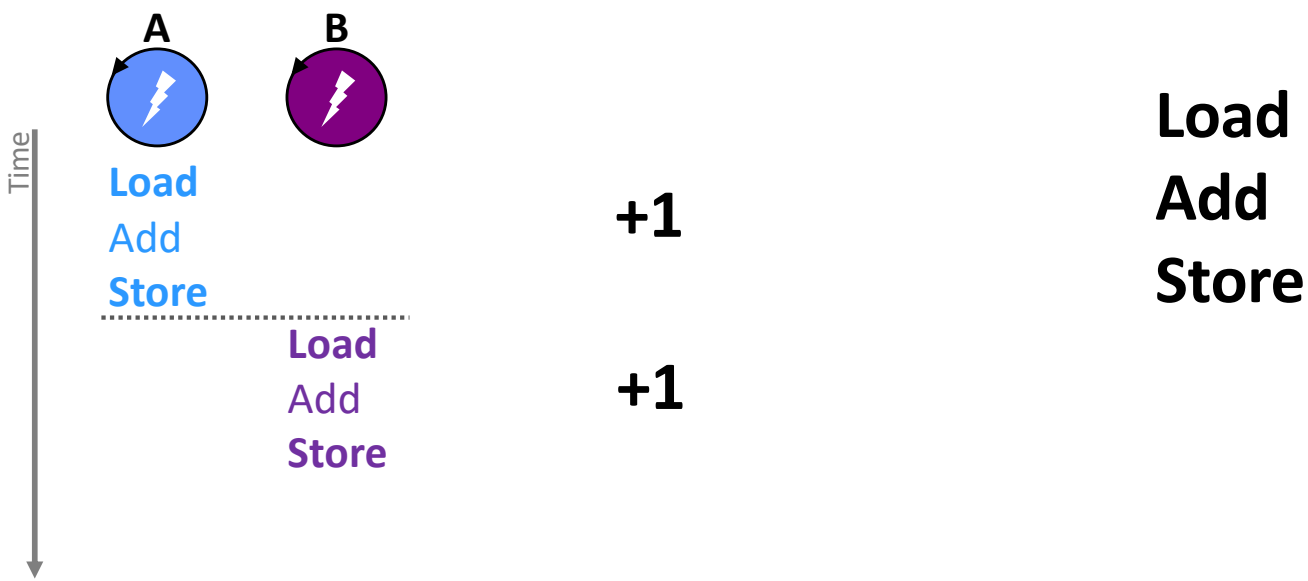


Reading Between the Lines (of C)

```
7a7:   8b 05 67 08 20 00      mov    0x200867(%rip),%eax
7ad:   83 c0 01               add    $0x1,%eax
7b0:   89 05 5e 08 20 00      mov    %eax,0x20085e(%rip)
```

Load
Add
Store

- Consider two threads (A, B) executing this code.

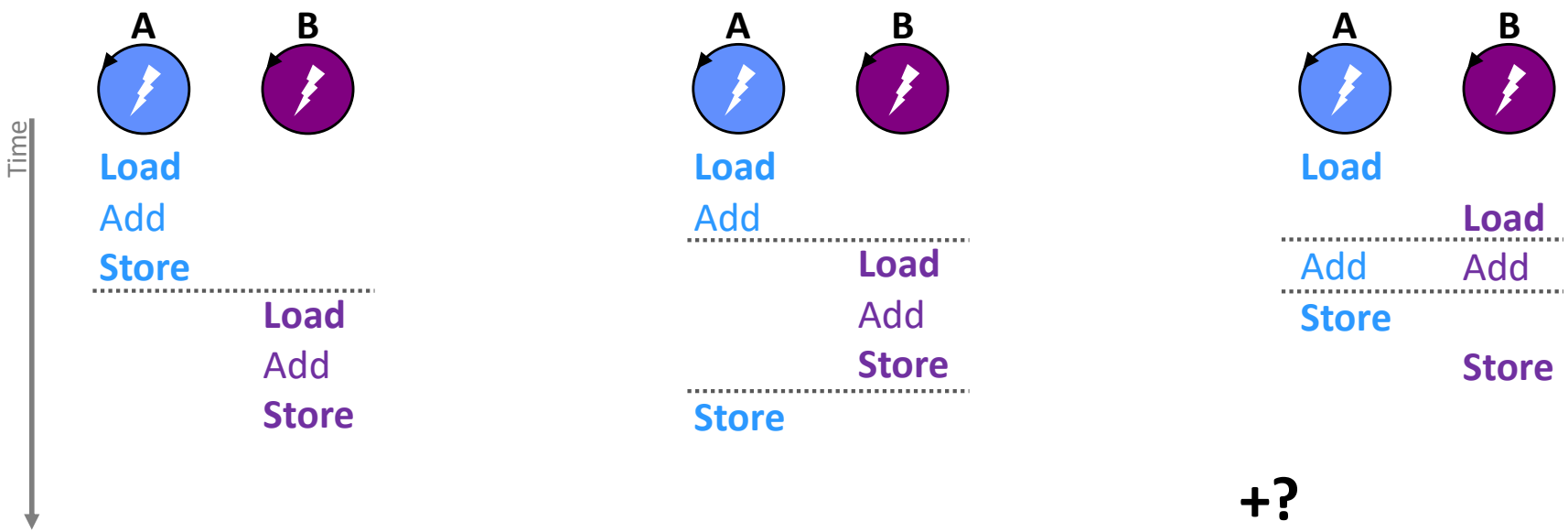


Reading Between the Lines (of C)

```
7a7:   8b 05 67 08 20 00      mov    0x200867(%rip),%eax
7ad:   83 c0 01               add    $0x1,%eax
7b0:   89 05 5e 08 20 00      mov    %eax,0x20085e(%rip)
```

Load
Add
Store

- What are possible interleavings for two threads (A, B)?



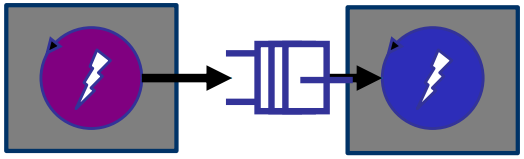
+?

Concurrency: Perils of shared state→mitigate

- Shared state→races. Synchronize! Locking and blocking.
- Do it right—but can we do less of it?
  - Less is more!
  - More concurrency?
  - Fewer bugs?

- <u>Go philosophy</u>: keep state private (once called "reactivist").

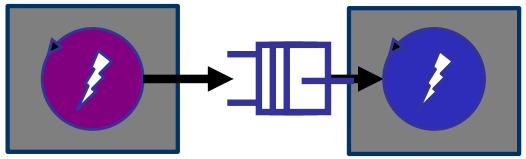
1. Decouple state across threads ("goroutines").
2. Place each under control of e.g. a single thread.
3. "Don't communicate by sharing memory, share memory by communicating".



Channels and goroutines

```
func main() {
        ch := make(chan int)

        go func() {
                ch <- 1
        }()
}
```

send

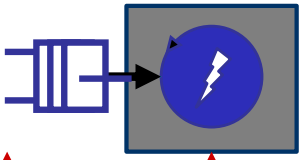


Idea: a goroutine controls the counter

```
incCh := make(chan struct{})
done := make(chan struct{})
go func() {
        for range incCh {
                counter++
        }
        done <- struct{}{}
}()
```

Cleanup

```
wg.Wait()
close(incCh)
<-done
```



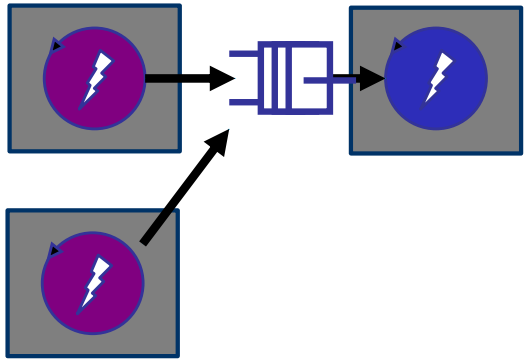
Only this thread increments counter.

Count threads send empty messages on this channel to request increments.

Two client threads request counts

```
func count(incCh chan<- struct{}, wg....) {
        defer wg.Done()
        for i := 0; i < 10_000_000; i++ {
                incCh <- struct{}{}
        }
}
```

```
go count(...)
go count(...)
```



Check for updates

Programming Techniques

S. L. Graham, R. L. Rivest
Editors

Communicating Sequential Processes

C.A.R. Hoare
The Queen's University
Belfast, Northern Ireland

Communications of the ACM

August 1978
Volume 21
Number 8

This paper suggests that input and output are basic primitives of programming and that parallel composition of communicating sequential processes is a fundamental program structuring method. When combined with a development of Dijkstra's guarded command, these concepts are surprisingly versatile. Their use is illustrated by sample solutions of a variety of familiar programming exercises.

Go uses a variant of the CSP model for concurrency.

Understanding Real-World Concurrency Bugs in Go

Abstract

Go is a statically-typed programming language that aims to provide a simple, efficient, and safe way to build multi-threaded software. Since its creation in 2009, Go has matured and gained significant adoption in production and open-source software. Go advocates for the usage of message passing as the means of inter-thread communication and provides several new concurrency mechanisms and libraries to ease multi-threading programming. It is important to understand the implication of these new proposals and the comparison of message passing and shared memory synchronization in terms of program errors, or bugs. Unfortunately, as far as we know, there has been no study on Go's concurrency bugs.

In this paper, we perform the first systematic study on concurrency bugs in real Go programs. We studied six popular Go software including Docker, Kubernetes, and gRPC. We analyzed 171 concurrency bugs in total, with more than half of them caused by non-traditional, Go-specific problems. Apart from root causes of these bugs, we also studied their fixes, performed experiments to reproduce them, and evaluated them with two publicly-available Go bug detectors. Overall, our study provides a better understanding on Go's concurrency models and can guide future researchers and practitioners in writing better, more reliable Go software and in developing debugging and diagnosis tools for Go.

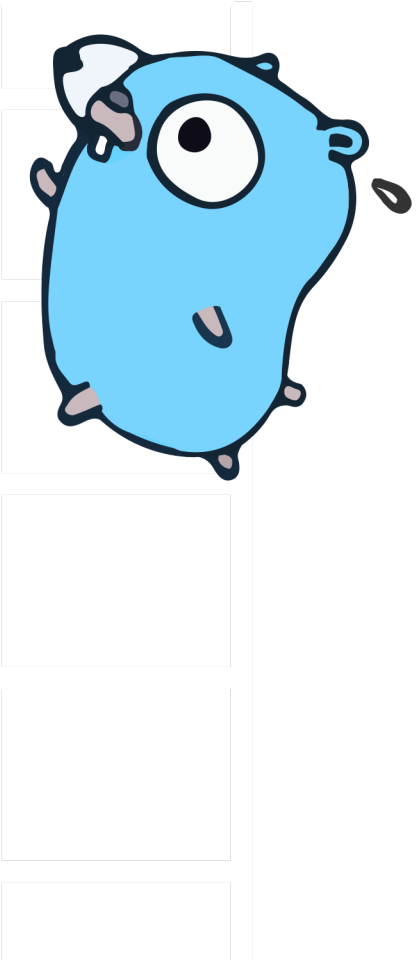
1 Introduction

Go [20] is a statically typed language originally developed by Google in 2009. Over the past few years, it has quickly gained attraction and is now adopted by many types of software in real production. These Go applications range from libraries [19] and high-level software [26] to cloud infrastructure software like container systems [13, 36] and key-value databases [10, 15].

A major design goal of Go is to improve traditional multi-threaded programming languages and make concurrent programming easier and less error-prone. For this purpose, Go centers its multi-threading design around two principles: 1) making threads (called *goroutines*) lightweight and easy to create and 2) using explicit messaging (called *channel*) to communicate across threads. With these design principles, Go proposes not only a set of new primitives and new libraries but also new implementation of existing semantics.

ACM Reference Format:
Tengfei Tu, Xiaoyu Liu, Linhai Song, and Yiying Zhang. 2019. Understanding Real-World Concurrency Bugs in Go . In *Proceedings of 2019 Architectural Support for Programming Languages and Operating Systems (ASPLOS'19).* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3297858.3304069

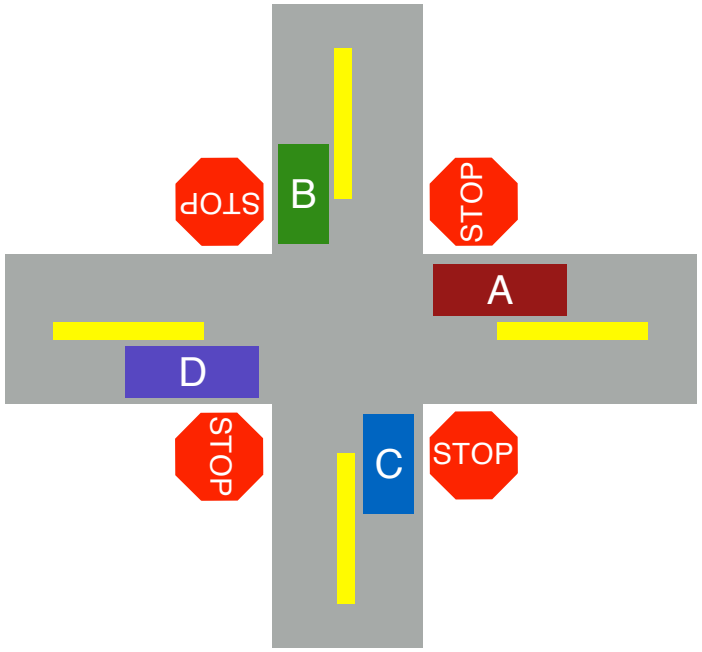
```
func main() {
        ch := make(chan int)

        go func() {
                ch <- 1
        }()

        go func() {
                <-ch
        }()

        select {
        case <-ch:
                fmt.Println("R")
        case ch <- 2:
                fmt.Println("S")
        }
}
```



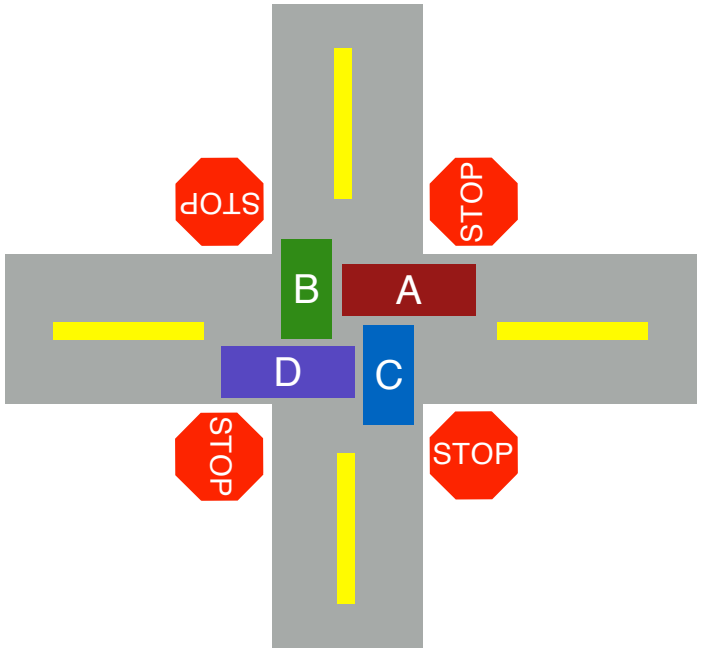
Deadlock!

```
% go run deadlock.go
r
s
fatal error: all goroutines are asleep - deadlock!


goroutine 1 [select]:
main.main()
/Users/chase/390/go-samples/go/src/deadlock.go:58 +0xf8
exit status 2
%
```

Traffic – Who goes?



Traffic – Deadlock! (Assuming no reverse…)



Select statement in Go

- For a fixed set of channel send/receive cases:
- Which case can proceed now without blocking?
  - Receive: input is available on channel
  - Send: buffer space is available on channel
- Picks one of those cases at **random**.
  - Communication proceeds
  - Execute associated statements
- None? Picks **default** case, if there is one.
  - Note: defaults in loops create a "hot loop" that burns CPU.
- Nothing ready? "**Park**" (sleep) thread until there is.

```
select {
case <-ch:
        fmt.Println("R")
case ch <- 2:
        fmt.Println("S")
}
```

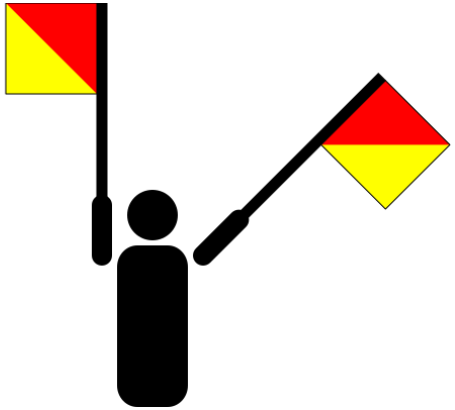
Two special cases of select

```
for {
    select {
        case c <- 0:
        case c <- 1:
    }
}

select {}
```

```
for {
    select {
        case c <- 0:
        case c <- 1:
    }
}

select {}
```

Semaphores

- A **semaphore** is an atomic counter:
  - Non-negative integer value
  - API: increment/decrement (V/P)
  - Sleep on decrement 0
  - Wake on increment 0
  - **Equivalent in power** to monitors
  - **Elegant!** But out of scope for us.



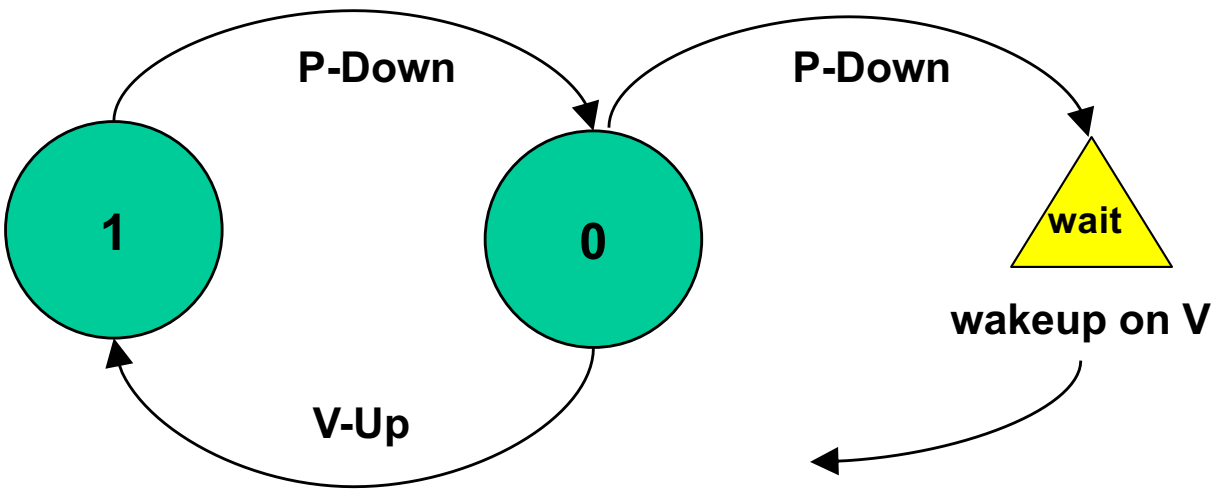
The name **semaphore** comes from a system for communicating with flags, used until the early 19th century. **Now you know.**

Curious?

http://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf

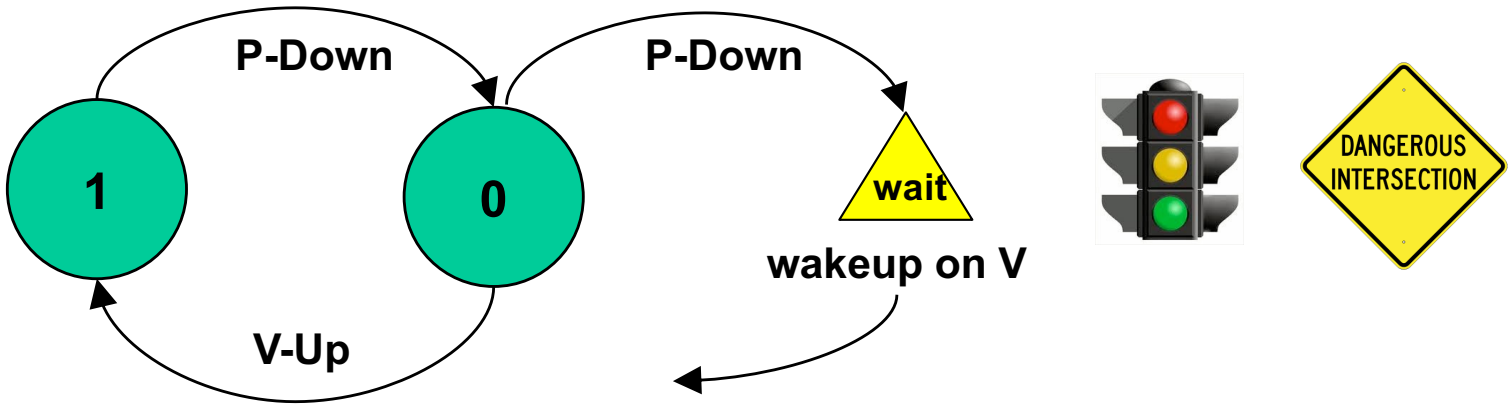
Example: binary semaphore

- A binary semaphore takes only values 0 and 1.
- It requires a usage constraint: the set of threads using the semaphore call P and V in strict alternation.



Mutual exclusion with a binary semaphore

- A mutex is just a binary semaphore with an initial value of 1, for which each thread calls P-V in strict pairs.
- Once a thread A completes its P, no other thread can P until A does a matching V.



Basic producer/consumer

```
empty->Init(1);
full->Init(0);
int buf;
```

```
void Produce(int m) {
        empty->P();
        buf = m;
        full->V();
}
```

```
int Consume() {
        int m;
        full->P();
        m = buf;
        empty->V();
        return(m);
}
```

Basic producer/consumer is called rendezvous: one producer, one consumer, and one item at a time. It is the same as ping-pong: producer and consumer access the buffer in strict alternation.

This use of a semaphore pair is called a split binary semaphore: the sum of the values is always one.